In many companies, employees who would normally spend their time in the office are occasionally required to visit clients or suppliers or attend a trade show or conference. Although these trips might be necessary, and on any given day there might be several people making such a trip, it may well be months or even years between such trips for any particular employee. Since it would therefore be rather expensive and impractical for each employee to have a company car, companies often offer a *car pool* instead; they have a limited number of cars that are available to all employees. When an employee needs to make an off-site trip, they book one of the pool cars for the appropriate time and return it for others to use when they return to the office. If there are no pool cars free on a given day, the employee will have to reschedule their trip for a subsequent date.

A *thread pool* is a similar idea, except that *threads* are being shared rather than cars. On most systems, it's impractical to have a separate thread for every task that can potentially be done in parallel with other tasks, but you'd still like to take advantage of the available concurrency where possible. A thread pool allows you to accomplish this; tasks that can be executed concurrently are submitted to the pool, which puts them on a queue of pending work. Each task is then taken from the queue by one of the *worker threads*, which executes the task before looping back to take another from the queue.

There are several key design issues when building a thread pool, such as how many threads to use, the most efficient way to allocate tasks to threads, and whether or not you can wait for a task to complete. In this section we'll look at some thread pool implementations that address these design issues, starting with the simplest possible thread pool.

9.1.1. The simplest possible thread pool

At its simplest, a thread pool is a fixed number of *worker threads* (typically the same number as the value returned by `std::thread::hardware_concurrency()`) that process work. When you have work to do, you call a function to put it on the queue of pending work. Each worker thread takes work off the queue, runs the